LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [5] – [Relasi antar Kelas]



NAMA: Lintang Anissa Hakim

NIM: 103132400040

KELAS: S1-SD-05

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (Object Oriented Programming atau OOP) adalah metode pemrograman yang menggunakan objek sebagai dasar dalam pembuatan program. Dalam OOP, program disusun menggunakan class dan object yang memiliki atribut serta method untuk menjalankan fungsi tertentu. Konsep ini membuat kode program lebih terstruktur dan mudah dipahami.Diagram kelas merupakan gambaran yang digunakan untuk menunjukkan struktur sebuah kelas dalam program. Diagram ini berisi nama kelas, atribut, method, serta hubungan antara satu kelas dengan kelas lainnya. Dengan adanya diagram kelas, programmer dapat lebih mudah memahami rancangan program sebelum mulai menulis kode. Association adalah hubungan antara dua kelas yang saling berinteraksi atau berkomunikasi. Pada hubungan ini, satu kelas dapat menggunakan method atau data dari kelas lain tanpa harus memiliki kelas tersebut. Contohnya adalah hubungan antara kelas Dosen dan Mahasiswa, ketika dosen memberikan nilai kepada mahasiswa. Aggregation adalah hubungan antara dua kelas yang menunjukkan bahwa suatu kelas memiliki atau menampung objek dari kelas lain. Namun, objek yang dimiliki tetap dapat berdiri sendiri meskipun objek yang menampungnya sudah tidak ada. Contohnya adalah hubungan antara kelas Jurusan dan Mahasiswa, karena mahasiswa tetap dapat ada meskipun jurusannya sudah tidak digunakan dalam program. Composition adalah hubungan antara dua kelas yang memiliki keterikatan kuat. Pada hubungan ini, objek bagian bergantung pada objek utama dan biasanya dibuat sebagai bagian dari objek tersebut. Contohnya adalah kelas Komputer yang terdiri dari CPU, Monitor, dan Mouse. Komponen tersebut dibuat dan dikelola sebagai bagian dari objek Komputer.Dalam bahasa pemrograman Java, hubungan antar kelas dapat diterapkan dengan membuat beberapa kelas yang saling berinteraksi. Setiap kelas memiliki atribut dan method sesuai fungsinya. Dengan memahami association, aggregation, dan composition, programmer dapat membuat program yang lebih terstruktur serta memahami cara objek saling berhubungan dalam sebuah sistem.

B. Guided

1. [Agregasi]

[Jurusan]

```
public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
```

```
        }
}
```

[Mahasiswa]

```
public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama) {
        this.NIM = NIM;
        this.Nama = Nama;
    }

    // Getter
    public String getNIM() {
        return NIM;
    }

    public String getNama() {
        return Nama;
    }

    // Setter
    public void setNIM(String NIM) {
        this.NIM = NIM;
    }

    public void setNama(String Nama) {
        this.Nama = Nama;
    }
}
```

[Main]

```
public class Main {
    public static void main(String[] args) {
        Jurusan jurusan = new Jurusan("TI", "Teknik Informatika");

        Mahasiswa m1 = new Mahasiswa("23001", "Andi");
        Mahasiswa m2 = new Mahasiswa("23002", "Budi");
        Mahasiswa m3 = new Mahasiswa("23003", "Citra");

        jurusan.addMahasiswa(m1);
        jurusan.addMahasiswa(m2);
        jurusan.addMahasiswa(m3);

        jurusan.displayMahasiswa();
    }
```

```
        }
```

Output :

```
] --- exec:3.6.3:exec (default-cli)
  Kode Jurusan: TI
  Nama Jurusan: Teknik Informatika
  Daftar Mahasiswa:
  1, 23001 Andi
  2, 23002 Budi
- 3, 23003 Citra
  ---------------------------------
  BUILD SUCCESS
  ---------------------------------
```

2. [Asosiasi]

[Dosen]

```
public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    //Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
         s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
         return s.getNilai();
    }

    public static void main(String[] args) {
```

```
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

[Mahasiswa]

```
[public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    //Setter
    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public String getNama() {
        return this.nama;
    }

    public void setNilai(int nilai){
        this.nilai = nilai;
    }

    //Getter
    public int getNilai(){
        return this.nilai;
    }


    public String getNim() {
        return this.nim;
    }
}
```

Output :

```
] --- exec:3.6.3:exec
  Nim :130118383
  Nama :Budi
  Nilai :90
  ------------------
  BUILD SUCCESS
```

3. [Komposisi]

[Processor]

```
public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

[Mouse]

```
public class Mouse {
    private String Merk, Type;

    public Mouse(String m, String t) {
        this.Merk = m;
        this.Type = t;
    }

    public void DisplaySpecMouse() {
        System.out.println(this.Merk + ", " + this.Type);
    }
}
```

[Monitor]

```
public class Monitor {
    private String Merk;

    public Monitor(String m) {
```

```
        this.Merk = m;
    }

    public void DisplaySpecMonitor() {
        System.out.println(this.Merk);
    }
}
```

[Komputer]

```
public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    // Constructor
    public Komputer() {
        processor = new Processor("Intel", 3000);
        monitor = new Monitor("Toshiba");
        mouse = new Mouse("Toshiba", "Wireless");
    }

    // Behaviour
    public void displaySpecKomputer() {
        System.out.println("Spesifikasi Laptop Toshiba");

        System.out.print("Processor: ");
        processor.displaySpec();

        System.out.print("Monitor: ");
        monitor.DisplaySpecMonitor();

        System.out.print("Mouse: ");
        mouse.DisplaySpecMouse();
    }

    // Main
    public static void main(String[] args) {
        Komputer komputer = new Komputer();
        komputer.displaySpecKomputer();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default
Spesifikasi Laptop Toshiba
Processor: Intel, 3000
Monitor: Toshiba
Mouse: Toshiba, Wireless
----------------------------
BUILD SUCCESS
----------------------------
```

C. Unguided

1. Soal Unguided 1

   Buat class RincianHarga dan ItemPesanan. Setiap item pesanan memiliki rincian harga yang berisi harga dasar dan pajak sebesar 10%. Objek RincianHarga harus dibuat langsung di dalam constructor ItemPesanan, bukan di dalam Main.

   Jawaban :

```
public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public double getPajak() {
        return pajak;
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

   Jawaban :

```
public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    // Constructor
    public ItemPesanan(String namaItem, double hargaDasar) {
```

```
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
    }
}
```

Penjelasan :
Pada class RincianHarga, terdapat dua atribut, yaitu hargaDasar dan pajak. Pajak dihitung sebesar 10% dari harga dasar, kemudian method getHargaTotal() digunakan untuk mendapatkan harga setelah pajak.
Sementara itu, class ItemPesanan menyimpan nama menu dan objek RincianHarga. Objek tersebut dibuat langsung menggunakan new RincianHarga(hargaDasar) di dalam constructor. Inilah yang menunjukkan komposisi, karena rincian harga dibuat sebagai bagian internal dari item pesanan. Kata kunci private digunakan agar atribut tidak bisa diakses langsung dari luar class. Data diakses melalui method getter.

2. Soal Unguided 2
   Buat class Pesanan yang menampung beberapa objek ItemPesanan menggunakan array. Objek ItemPesanan dibuat terlebih dahulu di Main, kemudian dimasukkan ke dalam Pesanan melalui method tambahItem()

   Jawaban :

```
public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    // Constructor
    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
            jumlahItem++;
```

```
            } else {
                System.out.println("Daftar pesanan sudah penuh.");
            }
        }

        public void tampilkanPesanan() {
            System.out.println("=== DAFTAR PESANAN CAFE ===");
            System.out.println("Nomor Meja: " + nomorMeja);

            for (int i = 0; i < jumlahItem; i++) {
                ItemPesanan item = daftarItem[i];
                RincianHarga rincian = item.getRincian();

                System.out.println(
                    (i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + rincian.getHargaTotal()
                    + " (Harga: " + rincian.getHargaDasar()
                    + ", Pajak: " + rincian.getPajak() + ")"
                );
            }
        }

        public int getNomorMeja() {
            return nomorMeja;
        }

        public ItemPesanan[] getDaftarItem() {
            return daftarItem;
        }

        public int getJumlahItem() {
            return jumlahItem;
        }
}
```

Penjelasan :
Pada class RincianHarga, terdapat dua atribut, yaitu hargaDasar dan pajak. Pajak dihitung sebesar 10% dari harga dasar, kemudian method getHargaTotal() digunakan untuk mendapatkan harga setelah pajak.
Sementara itu, class ItemPesanan menyimpan nama menu dan objek RincianHarga. Objek tersebut dibuat langsung menggunakan new RincianHarga(hargaDasar) di dalam constructor. Inilah yang menunjukkan komposisi, karena rincian harga dibuat sebagai bagian internal dari item pesanan. Kata kunci private digunakan agar atribut tidak bisa diakses langsung dari luar class. Data diakses melalui method getter.

3. Soal Unguided 3

Buat class `Kasir` yang memiliki atribut nama kasir dan method `prosesPembayaran(Pesanan pesanan)`.

Kasir menerima objek `Pesanan` melalui parameter method, menghitung total pembayaran, lalu mencetak struk.

Class `Kasir` tidak boleh menyimpan `Pesanan` sebagai atribut.

Jawaban :

```
public class Kasir {
    private String namaKasir;

    // Constructor
    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftarItem = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga rincian = daftarItem[i].getRincian();

            totalSubtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println();
        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja "
                + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : "
                + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Jawaban :

```
public class Main {
    public static void main(String[] args) {
```

```
        // Membuat objek kasir
        Kasir kasir = new Kasir("Budi");

        // Membuat objek item pesanan secara independen
        ItemPesanan item1 =
            new ItemPesanan("Iced Americano", 18000);

        ItemPesanan item2 =
            new ItemPesanan("Beef Croissant", 25000);

        ItemPesanan item3 =
            new ItemPesanan("Matcha Latte", 22000);

        // Membuat pesanan untuk Meja 4
        Pesanan pesanan = new Pesanan(4, 5);

        // Menambahkan item ke dalam pesanan
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // Menampilkan daftar pesanan
        pesanan.tampilkanPesanan();

        // Memproses pembayaran
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Modul5 ---
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
================================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Class Kasir bertugas menghitung total pembayaran berdasarkan semua item yang ada dalam pesanan.

Method prosesPembayaran() menerima objek Pesanan sebagai parameter,

lalu mengambil data item untuk menghitung total harga dasar, total pajak, dan total pembayaran.

Class Kasir tidak mempunyai atribut bertipe Pesanan. Objek pesanan hanya digunakan ketika method pembayaran dipanggil. Hal tersebut

menunjukkan asosiasi, yaitu hubungan interaksi antara dua objek yang tetap berdiri sendiri.

D. Kesimpulan

Program ini menerapkan tiga jenis relasi antar kelas dalam Java. Komposisi digunakan untuk menghubungkan item pesanan dengan rincian harganya, agregasi digunakan untuk menampung beberapa item dalam sebuah pesanan, dan asosiasi digunakan ketika kasir memproses pembayaran dari pesanan. Dengan menerapkan enkapsulasi dan array, program dapat mengelola data pesanan serta menghitung total pembayaran secara terstruktur.